

MANDALA COMPILER

Walter Alejandro Suárez Fonseca (20212020107)

Computer Science III

Ingeniería de Sistemas

Universidad Distrital Francisco José de Caldas

wasuarezf@udistrital.edu.co

Abstract

The Mandala Compiler is a software tool designed to interpret a domain-specific language (DSL) to define and generate artistic mandala patterns using geometric shapes. This project integrates fundamental compiler construction techniques—including lexical, syntactic, and semantic analysis—with creative generative art. Users can define mandalas using structured commands that specify shape types, sizes, colors, and repetitions. The compiled instructions are then rendered using Python's Tkinter library.

This document provides an in-depth overview of the Mandala Compiler, including its general structure, development process, supported syntax, and pattern definitions. It also outlines the formal grammar, key tokens, and lexemes used within the language. Additionally, a user guide is included to explain the available patterns, required attributes, syntax rules, common error-handling solutions, and extensibility considerations for future enhancements.

The Mandala Compiler serves as both an educational tool for learning compiler design principles and a creative platform for procedural art generation. Future improvements may involve expanding the

language to support more complex patterns, interactive customization, and potential applications in algorithmic art and computational design.

INTRODUCTION

MANDALAS

The Mandala Compiler is a software tool designed to interpret a domain-specific language (DSL) that allows users to define and generate mandala patterns using geometric shapes. The compiler processes the given code through lexical, syntactic, and semantic analysis, ultimately rendering the specified mandala using the Tkinter graphical interface.

What is a Mandala? A mandala is a geometric configuration of symbols, often circular, that holds spiritual, artistic, and meditative significance in various cultures. The term "mandala" originates from Sanskrit, meaning "circle." Traditionally, mandalas are used in Hinduism and Buddhism as spiritual guidance tools, representing the universe, balance, and harmony. In modern contexts, mandalas are used for relaxation, therapy, and artistic expression.[1] The Mandala Compiler allows users to generate unique mandala

patterns by defining different geometric patterns programmatically.

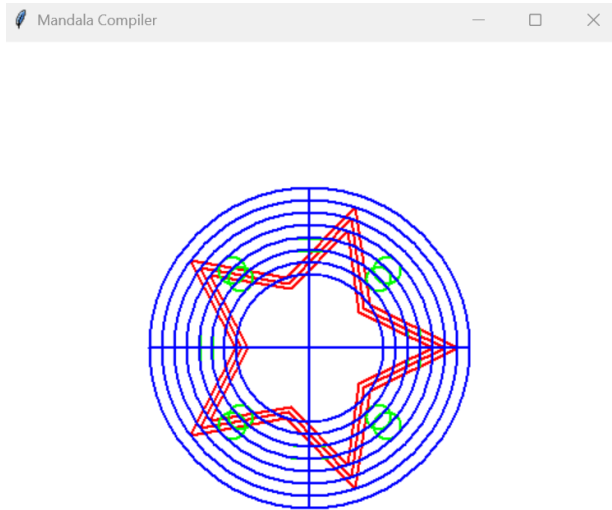


Image 1. Mandala Pattern

Justification: This project aims to provide a structured approach to generating artistic and symmetrical patterns using a compiler-like architecture. The motivation behind the Mandala Compiler is to blend computer science concepts—such as lexical, syntactic, and semantic analysis—with creative design. By implementing this project, students and developers can gain practical experience in compiler construction while exploring generative art.

Purpose of the program: The Mandala Compiler is designed to process a custom scripting language and generate visually appealing mandala patterns. It analyzes the input code through lexical, syntactic, and semantic processing to create structured, artistic designs. The goal is to provide an interactive and creative way to visualize structured patterns based on predefined rules.

Project Scope

This project focuses on:

- Implementing a compiler that parses custom input code.
- Supporting various geometric patterns, such as circles, diamonds, triangles, stars, and labyrinths.
- Using Tkinter for graphical representation.
- Providing customization options, such as size, color, repetition, and pattern variations.

Target Audience:

The project is intended for:

- Students and developers interested in compilers and graphical programming.
- Artists and designers who want to explore algorithm-generated art.
- Educators looking for a practical example of lexers, parsers, and semantic analysis in programming.

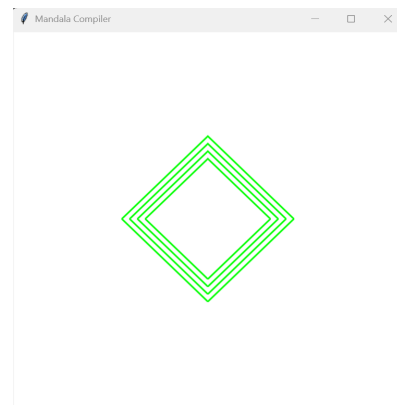


Image 2. Diamond Pattern

Conventions and Terminology Used:

- **Lexical Analysis:** Tokenizing input code using a lexical analyzer to identify keywords, symbols, and values.
- **Syntactic Analysis:** The process of checking if the tokens follow a valid structure.
- **Semantic Analysis:** The process of interpreting tokens to determine their meaning.

- **Pattern Types:** The various shapes that can be generated, such as circles, stars, or lotus flowers.
- **Tkinter Canvas:** The graphical interface where the mandala is drawn.

KEYWORDS

The Mandala Compiler uses a set of predefined keywords that dictate how the shapes are generated. Some of these keywords include:

- **START:** Marks the beginning of the mandala definition.
- **END:** Marks the end of the mandala definition.
- **PATTERN:** Defines a geometric pattern to be drawn.
- **SIZE:** Specifies the size of the pattern.
- **COLOR:** Determines the color of the pattern.
- **REPEAT:** Indicates how many times the pattern should be drawn.
- **LEVELS:** Used for patterns like MAZE to define complexity.
- **POINTS:** Defines the number of points for STAR patterns.
- **PETALS:** Defines the number of petals for LOTUS patterns.

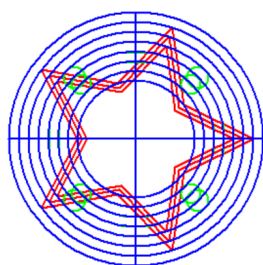
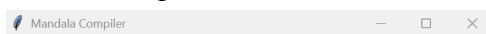


Image 3. Complex Mandala Pattern

- **Token:** A fundamental unit of the language, such as a keyword, identifier, number, or symbol.
- **Lexeme:** The actual text in the source code that corresponds to a token

Token Type	Example Lexeme	Description
KEYWORD	PATTERN	Defines a shape
NUMBER	50	Represents a size or count
COLO_CODE	#FF0000	Defines the color of a shape
IDENTIFIER	CIRCLE	Specifies the type of pattern

Generative Grammar:

The grammar for the Mandala Compiler follows a structured pattern:

```

<program> ::= START <statements> END
<statements> ::= <statement> | <statement>
<statements>
<statement> ::= PATTERN <pattern_type>
<attributes>
<pattern_type> ::= CIRCLE | DIAMOND |
STAR | TRIANGLE | MAZE | LOTUS | YIN
YANG
<attributes> ::= <attribute> | <attribute>
<attributes>
<attribute> ::= SIZE <number> | COLOR
<color_code> | REPEAT <number>

```



Image 4. Star Pattern

Tokens And Lexemes

Example Code

```

START
PATTERN CIRCLE SIZE 30 COLOR
#00FF00 REPEAT 10
PATTERN DIAMOND COLOR #FF0000
SIZE 50 REPEAT 2
PATTERN STAR POINTS 7 SIZE 100
COLOR #00FF00 REPEAT 10
PATTERN TRIANGLE COLOR #0000FF
SIZE 40 REPEAT 4
END

```

```

example_code2 = """
START
PATTERN YINYANG SIZE 5 COLOR #FF0000 REPEAT 5
PATTERN MAZE LEVELS 3 SIZE 50 COLOR #000000 REPEAT 3
PATTERN LOTUS PETALS 12 SIZE 200 COLOR #FF69B4 REPEAT 5
PATTERN CIRCLE SIZE 30 COLOR #00FF00 REPEAT 10
PATTERN DIAMOND COLOR #FF0000 SIZE 50 REPEAT 2
PATTERN STAR POINTS 7 SIZE 100 COLOR #00FF00 REPEAT 10
PATTERN TRIANGLE COLOR #0000FF SIZE 40 REPEAT 4
END
"""

```

Image 5. Input Example

Error Handling

The Mandala Compiler implements various error-handling mechanisms:

- **Lexical Errors:** If an unrecognized symbol is found, the compiler reports an invalid token.
- **Syntactic Errors:** If the input does not follow the expected grammar, an error message highlights the issue.
- **Semantic Errors:** If an invalid value (e.g., negative size or undefined pattern) is used, an exception is raised.

```

2. Realizando análisis sintáctico...
✗ Error Sintáctico: Error en línea 1: Atributo duplicado: SIZE
Token problemático: Token(SIZE, SIZE 30)
En la línea: 1

```

Image 6. Error Message Example

Example Error Message:

Syntax Error: Unexpected token 'PATERN' at line 2.

Did you mean 'PATTERN'?

```

1. Realizando análisis léxico...
✗ Error Léxico en línea 3: Token no reconocido: 'PATERN'
¿Quisiste decir 'PATTERN'?

```

Image 7. Lexical error message

Extensibility

Future enhancements to the Mandala Compiler may include:

- **New Shapes:** Adding more geometric patterns for greater artistic flexibility.
- **Export Options:** Enabling users to save mandalas as images or vector graphics.
- **Interactive Editing:** Implementing a graphical user interface (GUI) for real-time modifications.
- **Advanced Attributes:** Supporting gradient colors, transparency, and layer effects.

Mandala Compiler

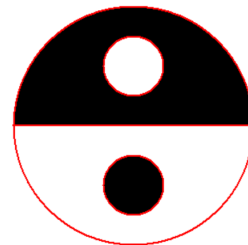


Image 8. YingYang Pattern

User Manual

Basic Structure:

Every program must follow this structure:

```

START
[pattern definitions]
END

```

- **START:** Indicates the beginning of the program
- **END:** Indicates the end of the program

- Between START and END, one or more patterns are defined

AVAILABLE PATTERNS

The compiler supports the following patterns:

1. CIRCLE: Simple circle
2. DIAMOND: Diamond shape
3. TRIANGLE: Triangle
4. SQUARE: Square
5. STAR: Star with a variable number of points
6. LOTUS: Lotus flower
7. MAZE: Maze pattern
8. PETALS_RADIATE: Radiating petals from the center
9. DOTS: Dots arranged in a circular pattern
10. YINYANG: Yin Yang symbol

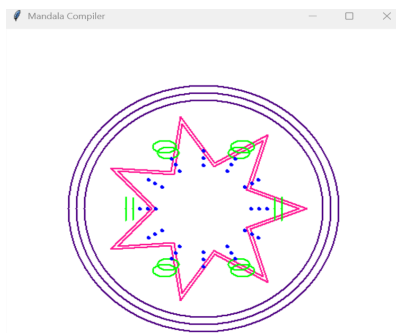


Image 9. Complex Mandala Pattern

General Attributes

Each pattern must include the following mandatory attributes:

- **SIZE:** Pattern size (positive number)
- **COLOR:** Color in hexadecimal format (e.g., #FF0000)
- **REPEAT:** Number of times the pattern is repeated (positive number)

Format:

PATTERN [type] SIZE [number] COLOR [hexcolor] REPEAT [number]

Specific Attributes

- **STAR:** Requires POINTS (number of points, minimum 3)

PATTERN STAR SIZE 85 COLOR #FFA500 REPEAT 3 POINTS 5

- **LOTUS:** Requires PETALS (number of petals)

PATTERN LOTUS SIZE 95 COLOR #FF1493 REPEAT 2 PETALS 8

- **MAZE:** Requires LEVELS (number of levels)

PATTERN MAZE SIZE 75 COLOR #4B0082 REPEAT 2 LEVELS 4

- **DOTS:** Requires COUNT (number of dots)

PATTERN DOTS SIZE 85 COLOR #DC143C REPEAT 3 COUNT 16

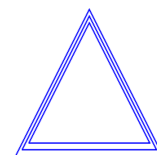
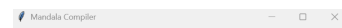


Image 10. Triangle Pattern

Usage Examples

- **Circle**
START PATTERN CIRCLE SIZE 100 COLOR #FF0000 REPEAT 3 END
- **5 Pointed Star**
START
PATTERN STAR SIZE 85 COLOR #FFA500 REPEAT 3 POINTS 5

END

- **Multiple Patterns**

```
START
PATTERN CIRCLE SIZE 150 COLOR
#4B0082 REPEAT 3
PATTERN STAR SIZE 120 COLOR
#FF1493 REPEAT 2 POINTS 7
PATTERN LOTUS SIZE 90 COLOR
#00FF00 REPEAT 2 PETALS 6
PATTERN DOTS SIZE 60 COLOR
#0000FF REPEAT 3 COUNT 12
END
```

Common Error Solutions

Error: "Missing SIZE/COLOR/REPEAT attribute"

- Ensure that all mandatory attributes are included for each pattern.
- Verify that they are written in uppercase.

Error: "Invalid color"

- Colors must be in hexadecimal format.
- They must start with # and have 6 hexadecimal characters.
- Correct example: #FF0000
- Incorrect example: FF0000

Error: "Invalid numeric value"

- All numeric values (SIZE, REPEAT, POINTS, etc.) must be positive.
- Negative numbers or zeros are not allowed.

Error: "Unrecognized pattern"

- Verify that the pattern name is spelled correctly and in uppercase.
- Check the list of available patterns.

Error: "Missing specific attribute"

- STAR requires POINTS

- LOTUS requires PETALS
- MAZE requires LEVELS
- DOTS requires COUNT

Error: "Tokens found after END"

- Ensure there is no additional code after the word END.
- All codes must be within START and END.

Suggestions

For best results:

- Maintain a consistent order of attributes
- Use reasonable values for SIZE (recommended: 50-150)
- Experiment with different pattern combinations
- Try different REPEAT values to create interesting effects

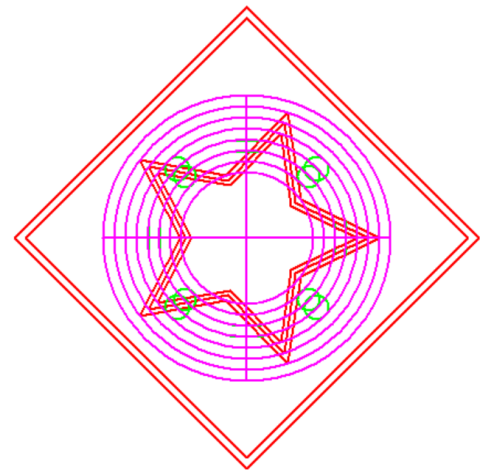
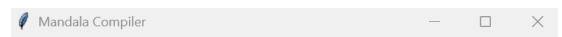


Image 11. Complex Mandala Pattern

Conclusion

The Mandala Compiler is a creative and technical project that bridges the gap between compiler design and generative art. Implementing lexical, syntactic, and semantic analysis, it allows users to programmatically define artistic patterns with structured syntax. Future improvements could include expanding the language with more complex patterns, adding user interactivity, or integrating machine learning for generative art variations.

BIBLIOGRAFÍA

[1] MANDALA. (n.d.). *Definition, history, types*. Retrieved from <https://www.britannica.com/topic/mandala-dia-gram>